

DataGrokr Git Assignment Part I

The assignment covers creating and initializing a Git repository, working with branches using the GitFlow workflow, creating and managing feature branches, staging and committing changes, merging branches, resolving merge conflicts, amending and dropping commits, squashing multiple commits, and pushing the local repository to GitHub. The following document presents the steps performed during the assignment along with the relevant Git commands and screenshots.

Final repository: <https://github.com/bhakthi133/my-exciting-project>

1. Repository Creation

The first step was to create a new project directory named **my-exciting-project** and initialize it as a Git repository.

```
mkdir my-exciting-project
cd my-exciting-project
git init
```

Description: The project directory was created and Git was initialized, creating a local .git repository for tracking the project.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git init
Initialized empty Git repository in C:/Users/Admin/Desktop/my-exciting-project/.git/
```

2. Setting the Default Branch

After initializing the Git repository, the default branch was renamed to **master** as required by the assignment.

```
git branch -M master
```

Description:

The command changes the name of the current branch to master.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git branch -M master
PS C:\Users\Admin\Desktop\my-exciting-project> git branch
PS C:\Users\Admin\Desktop\my-exciting-project> git status
On branch master

No commits yet
```

3. Creating the Initial Commit

Since the repository did not have any commits, an initial empty commit was created.

```
git commit --allow-empty -m "Initial commit"
```

Description:

An initial commit was created to establish the starting point of the repository.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git commit --allow-empty -m "Initial commit"
[master (root-commit) c5be7e0] Initial commit
PS C:\Users\Admin\Desktop\my-exciting-project> git branch
* master
```

4. Creating the Develop Branch and Initial Feature Branch

After creating the initial commit on the master branch, a develop branch was created from master.

After switching to the develop branch, a feature branch named **feature/initial-setup** was created for the initial project setup.

```
git branch develop
```

```
git checkout develop
```

```
git checkout -b feature/initial-setup
```

Description:

The develop branch was created and then checked out to continue the development work separately from the master branch. Also This creates a new feature branch from the current develop branch and immediately switches to it. The initial project files were then created on this feature branch.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout develop
Switched to branch 'develop'
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout -b feature/initial-setup
Switched to a new branch 'feature/initial-setup'
```

5. Creating the Project Files

On the feature/initial-setup branch, the required project files were created:

- **README** – contains This is a sample README.
- **LICENSE** – contains This is a sample LICENSE.
- **my-awesome-script.py** – contains the Python program.

Description:

The required files were created and the initial project content was added as specified in the assignment.

```
PS C:\Users\Admin\Desktop\my-exciting-project> echo "This is a sample README." > README
PS C:\Users\Admin\Desktop\my-exciting-project> echo "This is a sample LICENSE." > LICENSE
PS C:\Users\Admin\Desktop\my-exciting-project> echo "#!/bin/python"
>> print('Hello, World!') > my-awesome-script.py
```

6. Staging and Committing the Initial Setup

After creating the required files, they were added to the staging area and committed to the feature/initial-setup branch.

```
git add .
```

```
git commit -m "Initial setup"
```

Description:

git add . stages all the newly created files, and git commit saves the staged changes to the Git repository with the message "Initial setup".

```
PS C:\Users\Admin\Desktop\my-exciting-project> git add .
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Initial setup"
[feature/initial-setup 1202975] Initial setup
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 LICENSE
create mode 100644 README
create mode 100644 my-awesome-script.py
```

7. Merging the Initial Feature Branch into Develop

After committing the initial setup, the feature/initial-setup branch was merged into the develop branch.

```
git checkout develop
```

```
git merge feature/initial-setup
```

Description:

First, switched to the develop branch. Then, merged the changes from feature/initial-setup into develop.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout develop
Switched to branch 'develop'
PS C:\Users\Admin\Desktop\my-exciting-project> git merge feature/initial-setup
Updating c5be7e0..1202975
Fast-forward
 LICENSE          | Bin 0 -> 56 bytes
 README          | Bin 0 -> 54 bytes
 my-awesome-script.py | Bin 0 -> 78 bytes
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 LICENSE
create mode 100644 README
create mode 100644 my-awesome-script.py
```

8. Merging Develop into Master

After merging the initial feature branch into develop, the updated develop branch was merged into master.

```
git checkout master
```

```
git merge develop
```

Description:

Switched to the master branch and merged the changes from develop into master.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout master
Switched to branch 'master'
PS C:\Users\Admin\Desktop\my-exciting-project> git merge develop
Updating c5be7e0..1202975
Fast-forward
 LICENSE          | Bin 0 -> 56 bytes
 README          | Bin 0 -> 54 bytes
 my-awesome-script.py | Bin 0 -> 78 bytes
 3 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 LICENSE
 create mode 100644 README
 create mode 100644 my-awesome-script.py
```

9. Creating and Committing the First Enhancement

A new feature branch feature/enhancement-1 was created from develop. The Python script was then modified by changing the greeting from **Hello, World!** to **Howdy, World!**.

```
git checkout develop
```

```
git checkout -b feature/enhancement-1
```

The following line in my-awesome-script.py was changed:

Before:

```
print('Hello, World!')
```

After:

```
print('Howdy, World!')
```

The modified file was then staged and committed.

```
git add my-awesome-script.py
```

```
git commit -m "Enhanced greeting"
```

Description:

Created the feature/enhancement-1 branch, changed the greeting in the Python script to **Howdy, World!**, staged the modified file, and committed the change.

```

PS C:\Users\Admin\Desktop\my-exciting-project> git checkout develop
Switched to branch 'develop'
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout -b feature/enhancement-1
Switched to a new branch 'feature/enhancement-1'
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Enhanced greeting"
[feature/enhancement-1 2dfcb98] Enhanced greeting
1 file changed, 0 insertions(+), 0 deletions(-)

```

10. Creating the Second Enhancement Feature Branch

After completing the first enhancement, the develop branch was used to create the second feature branch, feature/enhancement-2.

```
git checkout develop
```

```
git checkout -b feature/enhancement-2
```

The greeting in my-awesome-script.py was changed from:

```
print('Howdy, World!')
```

to:

```
print('Hajimemashite sekai!')
```

The modified file was then staged and committed.

```
git add my-awesome-script.py
```

```
git commit -m "Added Japanese greeting"
```

Description:

Created feature/enhancement-2 from develop, changed the greeting to **Hajimemashite sekai!**, and committed the change.

```

PS C:\Users\Admin\Desktop\my-exciting-project> git checkout -b feature/enhancement-2
Switched to a new branch 'feature/enhancement-2'
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Added Japanese greeting"
[feature/enhancement-2 ca4c3dc] Added Japanese greeting
1 file changed, 0 insertions(+), 0 deletions(-)

```

11. Merging the First Enhancement into Develop

The feature/enhancement-1 branch was merged into the develop branch first.

```
git checkout develop
```

```
git merge feature/enhancement-1
```

Description:

Switched to develop and merged the changes from feature/enhancement-1 into it. This introduced the **Howdy, World!** version of the greeting into develop.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout develop
Switched to branch 'develop'
PS C:\Users\Admin\Desktop\my-exciting-project> git merge feature/enhancement-1
Updating 1202975..2dfcb98
Fast-forward
 my-awesome-script.py | Bin 78 -> 76 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
```

12. Merging the Second Enhancement and Resolving the Conflict

The feature/enhancement-2 branch was merged into develop. Since both enhancement branches modified the same line in my-awesome-script.py, a merge conflict occurred.

```
git merge feature/enhancement-2
```

The conflict was resolved by keeping:

```
print('Hajimemashite sekai!')
```

After resolving the conflict, the file was staged and the merge was committed.

```
git add my-awesome-script.py
```

```
git commit -m "Resolved merge conflict"
```

Description:

Merged feature/enhancement-2 into develop, resolved the conflict between the two greeting changes, and committed the resolved version.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git merge feature/enhancement-2
warning: Cannot merge binary files: my-awesome-script.py (HEAD vs. feature/enhancement-2)
Auto-merging my-awesome-script.py
CONFLICT (content): Merge conflict in my-awesome-script.py
Automatic merge failed; fix conflicts and then commit the result.
```

```
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Resolved merge conflict"
[develop aaa08b9] Resolved merge conflict
```

13. Amending Commits

First, switched to develop and created the new feature branch:

```
git checkout develop
```

```
git checkout -b feature/next-awesome-thing
```

Then added the following comment **below the shebang line** in my-awesome-script.py:

```
#!/bin/python
# This is an awesome Python script
print('Hajimemashite sekai!')
```

Shebang:

The **shebang** is the first line beginning with `#!` in a script. It specifies which interpreter should be used to run the script. Here, `#!/bin/python` indicates the Python interpreter.

Stage and commit the change:

```
git add my-awesome-script.py
git commit -m "Added a comment"
```

Now amend the last commit message:

```
git commit --amend -m "Added an awesome comment"
```

Description:

The last commit was amended by changing its commit message from **Added a comment** to **Added an awesome comment**.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout -b feature/next-awesome-thing
Switched to a new branch 'feature/next-awesome-thing'
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Added a comment"
[feature/next-awesome-thing 0ee0673] Added a comment
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\Admin\Desktop\my-exciting-project> git commit --amend -m "Added an awesome comment"
[feature/next-awesome-thing 8371c5d] Added an awesome comment
Date: Wed Sep 9 09:42:14 2026 +0530
1 file changed, 0 insertions(+), 0 deletions(-)
```

14. Dropping Commits

While still on the feature/next-awesome-thing branch, the last commit was removed using git reset.

```
git reset --hard HEAD~1
```

Description:

The `HEAD~1` refers to the commit immediately before the current commit. Using `--hard` moves the branch back by one commit and removes the changes introduced by the last commit from the working directory.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git reset --hard HEAD~1
HEAD is now at aaa08b9 Resolved merge conflict
```

15. Squashing Commits

The final three changes were made on a new branch called feature/part-1-finale. These three commits were then combined into a single commit.

Step 1: Switched to develop and created the feature branch

```
git checkout develop
```

```
git checkout -b feature/part-1-finale
```

```
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout develop
Switched to branch 'develop'
PS C:\Users\Admin\Desktop\my-exciting-project> git checkout -b feature/part-1-finale
Switched to a new branch 'feature/part-1-finale'
```

Step 2: Made the first modification

Opened my-awesome-script.py and changed the print statement to:

```
print('Hello! World, how is it going?.')
```

Stage and commit:

```
git add my-awesome-script.py
```

```
git commit -m "Update greeting 1"
```

```
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Update greeting 1"
[feature/part-1-finale 9585eb9] Update greeting 1
1 file changed, 0 insertions(+), 0 deletions(-)
```

Step 3: Made the second modification

Change the print statement again to:

```
print("Hello! World, hope you're doing well!.")
```

Stage and commit:

```
git add my-awesome-script.py
```

```
git commit -m "Update greeting 2"
```

```
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Update greeting 2"
[feature/part-1-finale 05d27ec] Update greeting 2
1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\Admin\Desktop\my-exciting-project> ^C
```

Step 4: Made the third modification

Change the print statement one final time to:

```
print('Hello! World, we are one!.')
```

Stage and commit:

```
git add my-awesome-script.py
git commit -m "Update greeting 3"
```

At this point, there will be **three separate commits**:

Update greeting 3

Update greeting 2

Update greeting 1

```
PS C:\Users\Admin\Desktop\my-exciting-project> git add my-awesome-script.py
PS C:\Users\Admin\Desktop\my-exciting-project> git commit -m "Update greeting 3"
[feature/part-1-finale 4a6c414] Update greeting 3
1 file changed, 0 insertions(+), 0 deletions(-)
```

Step 5: Squash the three commits

Run:

```
git rebase -i HEAD~3
```

An editor displayed something similar to:

```
pick <commit1> Update greeting 1
pick <commit2> Update greeting 2
pick <commit3> Update greeting 3
```

Changing it to:

```
pick <commit1> Update greeting 1
squash <commit2> Update greeting 2
squash <commit3> Update greeting 3
```

For the final commit message. Keeping:

Update greeting 1

Saving and closing again.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git rebase -i HEAD~3
[detached HEAD 0a7b9c2] Update greeting 1
Date: Wed Sep 9 09:43:32 2026 +0530
1 file changed, 0 insertions(+), 0 deletions(-)
Successfully rebased and updated refs/heads/feature/part-1-finale.
```

Step 6: Verified the squash

```
git log --oneline -4
```

There will be **one commit** representing the three greeting changes, instead of three separate commits.

Description:

The last three commits were combined into a single commit using interactive rebase and squash.

```
PS C:\Users\Admin\Desktop\my-exciting-project> git log --oneline -4
0a7b9c2 (HEAD -> feature/part-1-finale) Update greeting 1
aaa08b9 (feature/next-awesome-thing, develop) Resolved merge conflict
ca4c3dc (feature/enhancement-2) Added Japanese greeting
2dfcb98 (feature/enhancement-1) Enhanced greeting
```

16. Adding the Remote Repository

git remote add origin <https://github.com/bhakthi133/my-exciting-project.git>

->Pushing the Branches to GitHub

First pushing the master: git checkout master

git push -u origin master

Then pushing develop: git checkout develop

git push -u origin develop

Then pushing the feature branches: git push -u origin feature/initial-setup

git push -u origin feature/enhancement-1

git push -u origin feature/enhancement-2

git push -u origin feature/next-awesome-thing

git push -u origin feature/part-1-finale

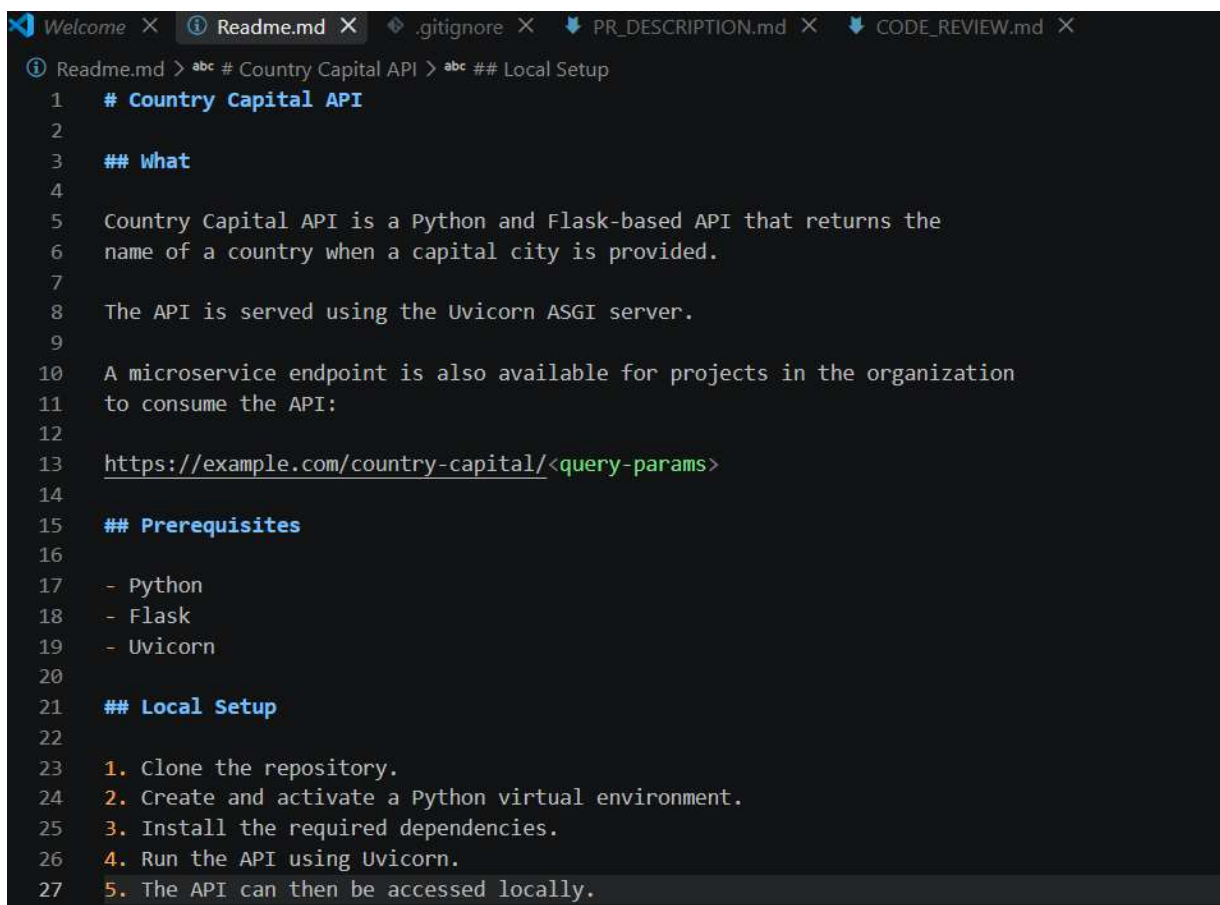
Description : The local repository was connected to the GitHub repository using the remote named origin. The required branches were then pushed to GitHub.

DataGrokr Git Assignment Part II

The assignment focuses on writing a good README, using .gitignore to keep the repository clean, creating clean pull request descriptions, and following review and approval best practices. The following document presents the solutions for each practice along with the relevant files and screenshots.

1. Writing a Good README

A README file was prepared for the **country-capital-api** repository based on the requirements given in the assignment. It describes what the project does, the prerequisites required, and the steps to set up the project locally.



```
1 # Country Capital API
2
3 ## What
4
5 Country Capital API is a Python and Flask-based API that returns the
6 name of a country when a capital city is provided.
7
8 The API is served using the Uvicorn ASGI server.
9
10 A microservice endpoint is also available for projects in the organization
11 to consume the API:
12
13 https://example.com/country-capital/<query-params>
14
15 ## Prerequisites
16
17 - Python
18 - Flask
19 - Uvicorn
20
21 ## Local Setup
22
23 1. Clone the repository.
24 2. Create and activate a Python virtual environment.
25 3. Install the required dependencies.
26 4. Run the API using Uvicorn.
27 5. The API can then be accessed locally.
```

2. Using .gitignore to Keep the Repository Clean

A .gitignore file was prepared to prevent specified files and directories from being tracked by Git.

.gitignore

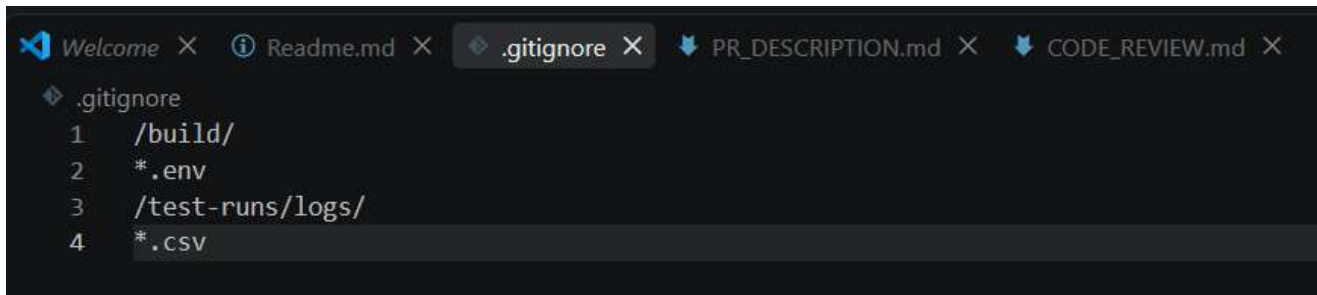
```
/build/
*.env
```

```
/test-runs/logs/
```

```
*.csv
```

Description:

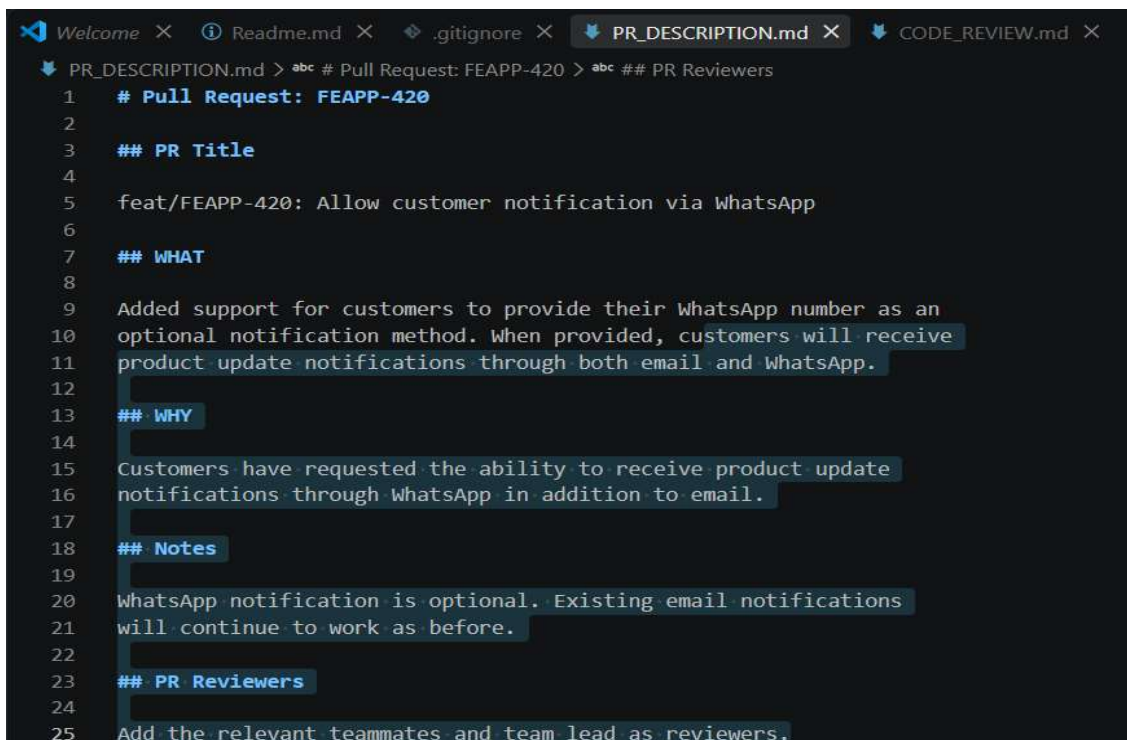
The .gitignore file ignores all files inside the root build directory, .env files, files inside test-runs/logs, and CSV files anywhere in the repository.



```
Welcome X Readme.md X .gitignore X PR_DESCRIPTION.md X CODE_REVIEW.md X
.gitignore
1 /build/
2 *.env
3 /test-runs/logs/
4 *.csv
```

3. Raising Clean Pull Requests

A PR description was prepared for the story **FEAPP-420: Allow customer notification via WhatsApp**.



```
Welcome X Readme.md X .gitignore X PR_DESCRIPTION.md X CODE_REVIEW.md X
PR_DESCRIPTION.md > abc # Pull Request: FEAPP-420 > abc ## PR Reviewers
1 # Pull Request: FEAPP-420
2
3 ## PR Title
4
5 feat/FEAPP-420: Allow customer notification via WhatsApp
6
7 ## WHAT
8
9 Added support for customers to provide their WhatsApp number as an
10 optional notification method. When provided, customers will receive
11 product update notifications through both email and WhatsApp.
12
13 ## WHY
14
15 Customers have requested the ability to receive product update
16 notifications through WhatsApp in addition to email.
17
18 ## Notes
19
20 WhatsApp notification is optional. Existing email notifications
21 will continue to work as before.
22
23 ## PR Reviewers
24
25 Add the relevant teammates and team lead as reviewers.
```

Description:

The PR description includes the title, WHAT, WHY, Notes, and PR Reviewers sections as specified in the assignment.

4. Review and Approval Best Practices

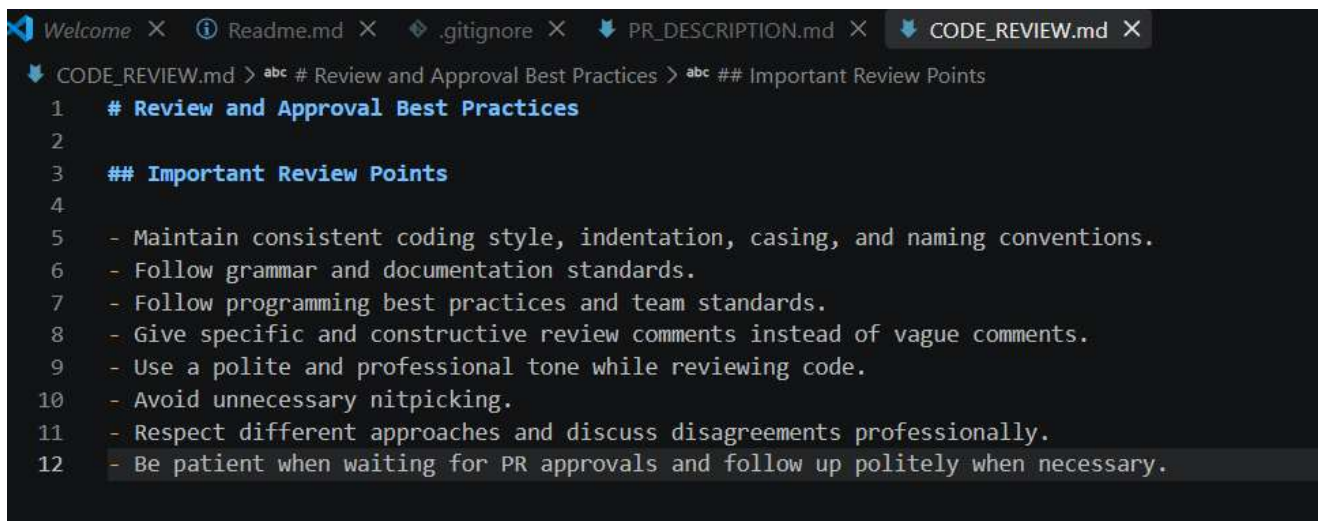
The following points were documented as important practices for reviewing and approving pull requests:

- Maintain consistent coding style, indentation, casing, and naming conventions.
- Follow grammar and documentation standards.
- Follow programming best practices and team standards.
- Give specific and constructive review comments instead of vague comments.
- Use a polite and professional tone while reviewing code.
- Avoid unnecessary nitpicking.
- Respect different approaches and discuss disagreements professionally.
- Be patient when waiting for PR approvals and follow up politely.

Description:

The review guidelines focus on maintaining code quality, providing specific and constructive feedback, using a professional tone, and handling differences in approaches respectfully.

Note : **Nitpicking** means focusing on **very small or minor mistakes/details**, especially when they are not very important.



```
CODE_REVIEW.md > abc # Review and Approval Best Practices > abc ## Important Review Points
1  # Review and Approval Best Practices
2
3  ## Important Review Points
4
5  - Maintain consistent coding style, indentation, casing, and naming conventions.
6  - Follow grammar and documentation standards.
7  - Follow programming best practices and team standards.
8  - Give specific and constructive review comments instead of vague comments.
9  - Use a polite and professional tone while reviewing code.
10 - Avoid unnecessary nitpicking.
11 - Respect different approaches and discuss disagreements professionally.
12 - Be patient when waiting for PR approvals and follow up politely when necessary.
```

